

Zaawansowane Techniki Optymalizacji

Problemy optymalizacji dyskretnej i ciągłej

prowadzący: dr inż. Jarosław Rudy

Poniżej zawarto definicje i inne przydatne informacje dotyczące problemów optymalizacji dyskretnej i ciągłej wykorzystywanych podczas zajęć. Pozostałe instrukcje laboratoryjne odwołują się do problemów poprzez ich numer np. 2.6 oznacza problem komiwojażera (TSP). Ponadto, w opisie problemów oznaczenia $\mathbb{N}_+$ oraz $\mathbb{N}_0$ oznaczają zbiór liczb naturalnych odpowiednio bez zera i z zerem. Podobnie $\mathbb{R}_+$ oznacza zbiór liczb rzeczywistych dodatnich (analogicznie $\mathbb{R}_+^2$ jest dwuwymiarowym zbiorem liczb rzeczywistych dodatnich).

1 Generator liczb pseudolosowych

Dla wszystkich laboratoriów poza tematem 1 (pierwsze 2 zajęcia) będzie istniała konieczność wygenerowania określonych instancji problemów. Instancje będziemy identyfikować za pomocą ich rozmiaru (jeden lub kilka parametrów, zależnie od problemu) oraz tzw. ziarna (seed). Sposób generacji instancji podany jest przy poszczególnych problemach. Do losowania danych będziemy korzystać z generatora liczb pseudolosowych zaopatrzonego w funkcje generacji następnej liczby całkowitej oraz następnej liczby rzeczywistej (z zadanych przedziałów obustronnie domkniętych). Generator korzysta z zadanego ziarna. Na stronie kursu umieszczone są kody źródłowe tego generatora dla czterech języków programowania: **python**, **C/C++**, **C#**, oraz **Java**. Nie należy losować instancji z użyciem innych generatorów. W dalszej części opisu przez $\text{init}(Z)$ rozumiemy inicjalizację generatora z użyciem ziarna Z , zaś przez $\text{nextInt}(a, b)$ oraz $\text{nextFloat}(a, b)$ losowanie odpowiednio liczby całkowitej i rzeczywistej z przedziału $[a, b]$.

2 Problemy optymalizacji dyskretnej

Rozdział ten zawiera definicje problemów dyskretnych używanych przez większość zajęć. Wszystkie problemy należą do kategorii jednokryterialnych. Dokładny sposób definicji może się różnić od tego znanego z literatury, ale powinien reprezentować dokładnie te same problemy. Ponadto, problemy należy traktować ogólnie tzn. należy zakładać dowolne możliwe dane, nawet jeśli omówione powyżej generatory dostarczają wąskiego i specyficznego typu instancji.

2.1 Problem sumy podzbioru (Subset Sum Problem)

Dany jest zbiór $S = \{s_1, s_2, \dots, s_n\}$ zawierający $n \in \mathbb{N}_+$ liczb, gdzie $s_i \in \mathbb{N}_+$. Dana jest też wartość $T \in \mathbb{N}_+$. Jako rozwiązanie należy znaleźć taki podzbiór X zbioru S aby suma jego elementów była możliwie bliska wartości T tj.:

$$\min_{X \in 2^S} |T - \sum_{x \in X} x|, \quad (1)$$

gdzie 2^S to zbiór potęgowy S , czyli zbiór wszystkich podzbiorów S .

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. $s_i \leftarrow \text{nextInt}(0, 100)$.
3. $T \leftarrow \text{nextInt}(0, 50n)$.

2.2 Zagadnienie transportowe (Transportation Problem)

Dane jest $n \in \mathbb{N}_+$ dostawców i $m \in \mathbb{N}_+$ odbiorców (uwaga, możliwe że $n \neq m$!). Podaż (produkcja) dostawcy i wynosi $S_i \in \mathbb{N}_+$, zaś popyt (zapotrzebowanie) odbiorcy j wynosi $D_j \in \mathbb{N}_+$. Dodatkowo wiemy, że suma popytów równa jest sumie podaży:

$$\sum_{i=1}^n S_i = \sum_{j=1}^m D_j. \quad (2)$$

Koszt transportu 1 jednostki towaru od dostawcy i do odbiorcy j wynosi $k_{ij} \in \mathbb{N}_+$. Należy dla każdej pary (i, j) określić wartość $x_{ij} \in \mathbb{N}_0$ (liczba jednostek transportowanych z i do j), tak by zminimalizować łączny koszt transportu. Innymi słowy:

$$\text{minimize} \quad \sum_{i=1}^n \sum_{j=1}^m x_{ij} k_{ij}, \quad (3)$$

$$\text{subject to} \quad \sum_{i=1}^n x_{ij} = D_j \quad \text{dla } j = 1, 2, \dots, m, \quad (4)$$

$$\sum_{j=1}^m x_{ij} = S_i \quad \text{dla } i = 1, 2, \dots, n. \quad (5)$$

Generacja instancji Dla parametrów n i m oraz ziarna Z :

1. $\text{init}(Z)$.
2. $s \leftarrow d \leftarrow 0$.
3. Dla i od 1 do n :
 - 3.1. $S_i \leftarrow \text{nextInt}(1, 20)$.
 - 3.2. $s \leftarrow s + S_i$.
4. Dla j od 1 do m :
 - 4.1. $D_j \leftarrow \text{nextInt}(1, 20)$.
 - 4.2. $d \leftarrow d + D_j$.
5. Dopóki $s < d$:
 - 5.1. $i \leftarrow \text{nextInt}(1, n)$.
 - 5.2. $S_i \leftarrow S_i + 1$.
 - 5.3. $s \leftarrow s + 1$.

6. Dopóki $d < s$:
 - 6.1. $j \leftarrow \text{nextInt}(1, m)$.
 - 6.2. $D_j \leftarrow D_j + 1$.
 - 6.3. $d \leftarrow d + 1$.
7. Dla i od 1 do n :
 - 7.1. Dla j od 1 do m :
 - 7.1.1. $k(i, j) \leftarrow \text{nextInt}(1, 50)$.

2.3 Zagadnienie przydziału (Assignment Problem)

Dane jest $n \in \mathbb{N}_+$ zadań i tyle samo wykonawców. Koszt wykonania zadania i przez wykonawcę j dany jest przez wartość $k(i, j) \in \mathbb{N}_+$. Celem zadania jest określenie przydziału zadań do wykonawców (każde zadanie ma dokładnie jednego wykonawcę i każdy wykonawca ma jedno zadanie), tak by zminimalizować koszt wykonania wszystkich zadań:

$$\min_f \sum_{i=1}^n k(i, f(i)), \quad (6)$$

gdzie f jest funkcją przydziału tj. $f(i)$ jest wykonawcą, do którego przydzielono zadanie i .

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. Dla j od 1 do n :
 - 2.1.1. $k(i, j) \leftarrow \text{nextInt}(1, 50)$.

2.4 Kwadratowe zagadnienie przydziału (Quadratic Assignment Problem)

Dane jest $n \in \mathbb{N}_+$ zakładów i tyle samo lokalizacji. Dla każdej pary zakładów (i, j) określony jest przepływ $w(i, j) \in \mathbb{N}_+$. Podobnie dla każdej pary lokalizacji (k, l) określona jest odległość $d(k, l) \in \mathbb{N}_+$. Należy określić przydział zakładów do lokalizacji (każdy zakład ma mieć dokładnie jedną lokalizację i na odwrót), tak by minimalizować koszt dany jako suma iloczynów przepływów i odległości zakładów:

$$\min \sum_{i=1}^n \sum_{j=1, j \neq i}^n w(i, j) d(f(i), f(j)), \quad (7)$$

gdzie f jest przydziałem lokalizacji tj. $f(i)$ jest lokalizacją, do której przydzielono zakład i . Warunek $j \neq i$ nie jest ściśle potrzebny, gdyż ze względu na sposób generacji instancji poniżej zawsze zachodzi $w(i, i) = 0$ oraz $d(f(i), f(i)) = 0$.

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. Dla j od 1 do n :
 - 2.1.1. Jeżeli $i \neq j$ to $w(i, j) \leftarrow \text{nextInt}(1, 50)$, w przeciwnym razie $w(i, j) \leftarrow 0$.
3. Dla k od 1 do n :
 - 3.1. Dla l od 1 do n :
 - 3.1.1. Jeżeli $k \neq l$ to $d(k, l) \leftarrow \text{nextInt}(1, 50)$, w przeciwnym razie $d(k, l) \leftarrow 0$.

2.5 Dyskretny problem plecakowy (Discrete Knapsack Problem)

Dany jest zbiór $P = \{1, 2, \dots, n\}$ zawierający $n \in \mathbb{N}_+$ przedmiotów. Dla każdego przedmiotu $i \in P$ dana jest jego wartość $c_i \in \mathbb{N}_+$ oraz waga $w_i \in \mathbb{N}_+$. Oprócz tego dana jest pojemność plecaka $B \in \mathbb{N}_+$. W podstawowej wersji zadanie polega na wyznaczeniu takiego podzbioru X zbioru P , że suma wag przedmiotów z X jest nie większa niż B zaś suma ich wartości jest maksymalna tj.:

$$\text{maximize} \quad \sum_{p \in P} x_p c_p, \quad (8)$$

$$\text{subject to} \quad \sum_{p \in P} x_p w_p \leq B, \quad (9)$$

gdzie $x_i = 1$ jeśli przedmiot i jest w plecaku oraz $x_i = 0$ w przeciwnym wypadku.

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. $S \leftarrow 0$:
3. Dla i od 1 do n :
 - 3.1. $c_i \leftarrow \text{nextInt}(1, 10)$.
 - 3.2. $w_i \leftarrow \text{nextInt}(1, 10)$.
 - 3.3. $S \leftarrow S + w_i$.
4. $B \leftarrow \text{nextInt}(\lfloor \frac{S}{4} \rfloor, \lfloor \frac{S}{2} \rfloor)$.

2.6 Problem komiwojażera (Travelling Salesman Problem)

Dane jest $n \in \mathbb{N}_+$ miast. Odległość od miasta i do miasta j dana jest przez $d_{ij} \in \mathbb{N}_+$, przy czym nie musi zachodzić $d_{ij} = d_{ji}$, zaś odległości postaci d_{ii} (z danego miasta do siebie samego) są nieistotne. Należy określić kolejność (permutację) odwiedzania miast, tak by zminimalizować długość cyklu Hamiltona:

$$\min_f d(f(n), f(1)) + \sum_{i=1}^{n-1} d(f(i), f(i+1)), \quad (10)$$

gdzie f jest kolejnością miast tj. $f(i)$ jest miastem, które należy odwiedzić jako i -te.

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. Dla j od 1 do n :
 - 2.1.1. Jeżeli $i \neq j$ to $d_{ij} \leftarrow \text{nextInt}(1, 30)$, w przeciwnym razie $d_{ij} \leftarrow 0$.

2.7 Szeregowanie zadań na jednej maszynie z ważoną sumą opóźnień (Scheduling on a single machine with total weighted tardiness, $1||w_i T_i$)

Dane jest $n \in \mathbb{N}_+$ zadań. Zadanie i ma czas wykonania równy $p_i \in \mathbb{N}_+$, wagę $w_i \in \mathbb{N}_+$ oraz pożądaný termin zakończenia (deadline) równy $d_i \in \mathbb{N}_+$. Należy określić kolejność wykonywania zadań π na maszynie, tak aby zminimalizować ważoną sumę spóźnień:

$$\min_{\pi} \sum_{i=1}^n w_{\pi(i)} T_{\pi(i)}, \quad (11)$$

gdzie π jest kolejnością wykonywania zadań tj. $\pi(i)$ określa które zadanie będzie wykonywane jako i -te na maszynie. Z kolei $T_{\pi(i)}$ jest spóźnieniem zadania które jest wykonywane jako i -te. Spóźnienie to dane jest wzorem:

$$T_{\pi(i)} = \max\{0, C_{\pi(i)} - d_{\pi(i)}\}, \quad (12)$$

gdzie $C_{\pi(i)}$ jest czasem zakończenia zadania wykonywanego jako i -te, które można obliczyć rekurencyjnie zaczynając od $i = 1$ (gdyż czas zakończenia zadania zależy od czasu zakończenia poprzednio wykonywanego zadania) zgodnie z poniższymi wzorami:

$$C_{\pi(1)} = p_{\pi(1)}, \quad (13)$$

$$C_{\pi(i)} = C_{\pi(i-1)} + p_{\pi(i)} \quad \text{dla } i > 1, \quad (14)$$

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. $S \leftarrow 0$.
3. Dla i od 1 do n :
 - 3.1. $w_i \leftarrow \text{nextInt}(1, 10)$.
 - 3.2. $p_i \leftarrow \text{nextInt}(1, 100)$.
 - 3.3. $S \leftarrow S + p_i$.
4. Dla i od 1 do n :
 - 4.1. $d_i \leftarrow \text{nextInt}(\lfloor \frac{S}{4} \rfloor, \lfloor \frac{S}{2} \rfloor)$.

2.8 Permutacyjny problem przepływowy (Permutation Flow Shop Problem, $PF||C_{\max}$)

Dane jest $n \in \mathbb{N}_+$ zadań i $m \in \mathbb{N}_+$ maszyn. Dla ułatwienia przyjmujemy że $m = 3$. Czas wykonywania zadania j na maszynie i wynosi $p_j^i \in \mathbb{N}_+$.

Zadania należy wykonać po kolei na wszystkich maszynach tj. najpierw na maszynie 1 potem na 2 i na końcu na 3 (reprezentuje to tzw. kolejność technologiczną określoną dla danego procesu produkcyjnego). Z kolei każda maszyna wykonuje zadania w tej samej kolejności (kolejność maszynowa), która z kolei jest zmienną decyzyjną tj. kolejność maszynową należy dobrać. Dodatkowo występują ograniczenia:

- zadanie należy zakończyć na maszynie i zanim będzie można je rozpocząć na maszynie $i + 1$,
- maszyna może wykonywać co najwyżej jedno zadanie naraz,
- wykonywanych zadań nie wolno przerywać ani wywłaszczać.

Należy określić kolejność wykonywania zadań π (tj. $\pi(j)$ określa które zadanie będzie wykonywane na maszynach jako j -te), tak aby zminimalizować długość uszeregowania (tzw. makespan) $C_{\max}$, czyli czas zakończenia wszystkich zadań:

$$\min_{\pi} C_{\max} = \min_{\pi} \max_j C_{\pi(j)}^m, \quad (15)$$

gdzie $C_{\pi(j)}^m$ jest czasem zakończenia zadania wykonywanego jako j -te. Można to uprościć, gdyż wiemy że najpóźniej zakończy się zadanie $\pi(n)$ (wynika to ze wzorów (17)–(20) poniżej) wobec czego:

$$C_{\max} = C_{\pi(n)}^m = C_{\pi(n)}^3. \quad (16)$$

Aby obliczyć czas zakończenia ostatniego zadania, należy obliczyć pozostałe czasy zakończenia, gdyż czas zakończenia $C_{\pi(j)}^i$ zadania $\pi(j)$ na maszynie i zależy nie tylko od samego czasu p_j^i ale od czasu zakończenia poprzednika maszynowego (zadania wykonywanego wcześniej na tej samej maszynie czyli $C_{\pi(j-1)}^i$) oraz poprzednika technologicznego (czasu zakończenia wykonywania tego zadania na maszynie poprzedniej czyli $C_{\pi(j)}^{i-1}$). Wyjątkiem są zadania wykonywane na pierwszej maszynie ($i = 1$) oraz zadania wykonywane na maszynie jako pierwsze ($j = 1$), gdyż nie mają one jednego z poprzedników (lub nie mają obu). Ostatecznie czasy zakończenia można obliczyć korzystając z poniższych rekurencyjnych wzorów:

$$C_{\pi(j)}^i = \max\{C_{\pi(j)}^{i-1}, C_{\pi(j-1)}^i\} + p_{\pi(j)}^i, \quad \text{dla } j > 1 \wedge i > 1, \quad (17)$$

$$C_{\pi(j)}^i = C_{\pi(j)}^{i-1} + p_{\pi(j)}^i, \quad \text{dla } j = 1 \wedge i > 1, \quad (18)$$

$$C_{\pi(j)}^i = C_{\pi(j-1)}^i + p_{\pi(j)}^i, \quad \text{dla } j > 1 \wedge i = 1, \quad (19)$$

$$C_{\pi(j)}^i = p_{\pi(j)}^i, \quad \text{dla } j = 1 \wedge i = 1. \quad (20)$$

Uwaga! Licząc czas $C_{\pi(j)}^i$ należy mieć pewność, że czasy $C_{\pi(j-1)}^i$ oraz $C_{\pi(j)}^{i-1}$ są już policzone. Najlepiej liczyć czasy w kolejności:

$$\begin{aligned} C_{\pi(1)}^1 &\rightarrow C_{\pi(2)}^1 \rightarrow \dots \rightarrow C_{\pi(n)}^1 \rightarrow \\ &\rightarrow C_{\pi(1)}^2 \rightarrow C_{\pi(2)}^2 \rightarrow \dots \rightarrow C_{\pi(n)}^2 \rightarrow \\ &\rightarrow C_{\pi(1)}^3 \rightarrow C_{\pi(2)}^3 \rightarrow \dots \rightarrow C_{\pi(n)}^3. \end{aligned} \quad (21)$$

Generacja instancji Dla parametru n i m oraz ziarna Z :

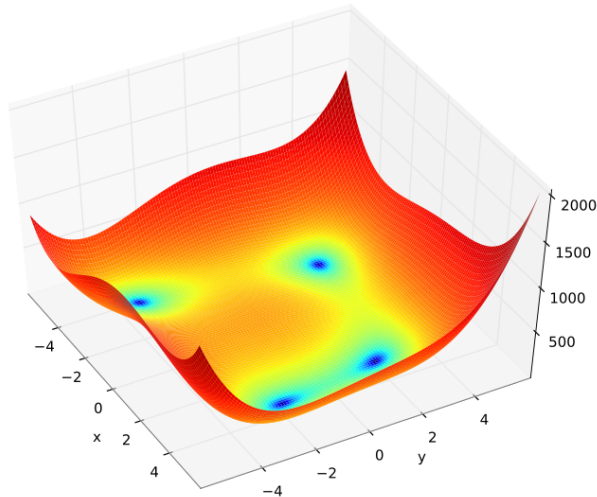
1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. Dla j od 1 do 3:
 - 2.1.1. $p_j^i \leftarrow \text{nextInt}(1, 99)$.

3 Funkcje ciągłe dwóch zmiennych

Poniżej zawarto definicje problemów optymalizacji ciągłej w postaci funkcji dwóch zmiennych x_1 oraz x_2 . Każdy opis zawiera wzór, dziedzinę (zakres obu zmiennych), rozwiązania optymalne (wraz z wartością funkcji celu) oraz poglądowy rysunek przedstawiający wykres danej funkcji.

3.1 Himmelblau's function

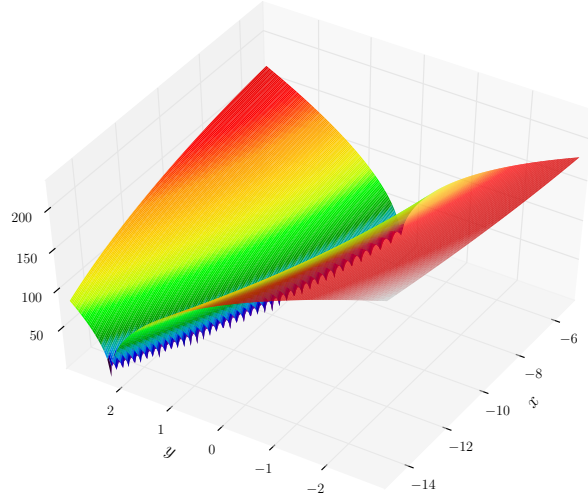
$$\begin{aligned} \text{minimize} \quad & f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \\ \text{domain} \quad & x_1 \in [-5, 5], \quad x_2 \in [-5, 5], \\ \text{optima} \quad & f(3, 2) = 0, \\ & f(-2.805118, 3.131312) = 0, \\ & f(-3.77931, -3.283186) = 0, \\ & f(3.584428, -1.848126) = 0. \end{aligned} \tag{22}$$



Rysunek 1: Himmelblau's function

3.2 Bukin function N.6

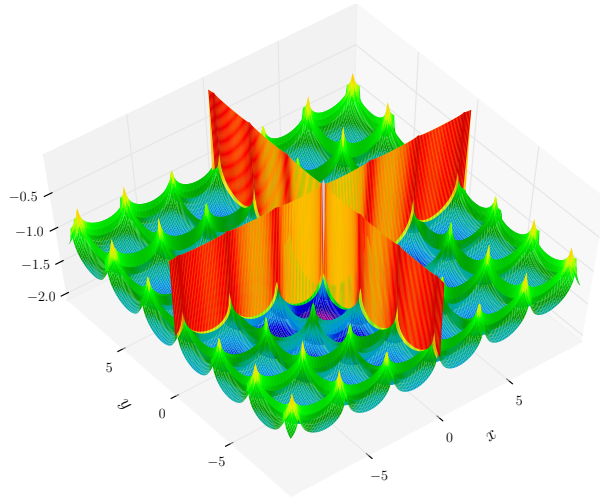
$$\begin{aligned}
 \text{minimize} \quad & f(x_1, x_2) = 100\sqrt{|x_2 - 0.01x_1^2|} + 0.01|x_1 + 10|, \\
 \text{domain} \quad & x_1 \in [-15, -5], \quad x_2 \in [-3, 3], \\
 \text{optimum} \quad & f(-10, 1) = 0.
 \end{aligned} \tag{23}$$



Rysunek 2: Bukin function N.6

3.3 Cross-in-Tray function

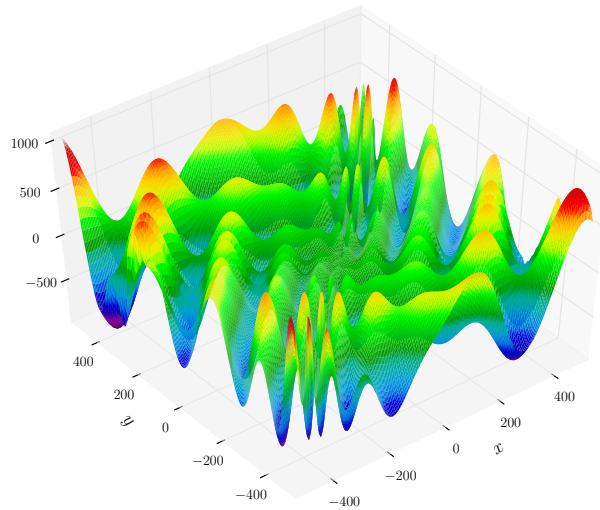
$$\begin{aligned}
 \text{minimize} \quad & f(x_1, x_2) = -0.0001 \left[\left| \sin(x_1) \sin(x_2) \exp \left(\left| 100 - \frac{\sqrt{x_1^2 + x_2^2}}{\pi} \right| \right) + 1 \right| \right]^{0.1}, \\
 \text{domain} \quad & x_1 \in [-10, 10], \quad x_2 \in [-10, 10], \\
 \text{optima} \quad & f(1.34941, -1.34941) = -2.06261, \\
 & f(1.34941, 1.34941) = -2.06261, \\
 & f(-1.34941, 1.34941) = -2.06261, \\
 & f(-1.34941, -1.34941) = -2.06261.
 \end{aligned} \tag{24}$$



Rysunek 3: Cross-in-Tray function

3.4 Eggholder function

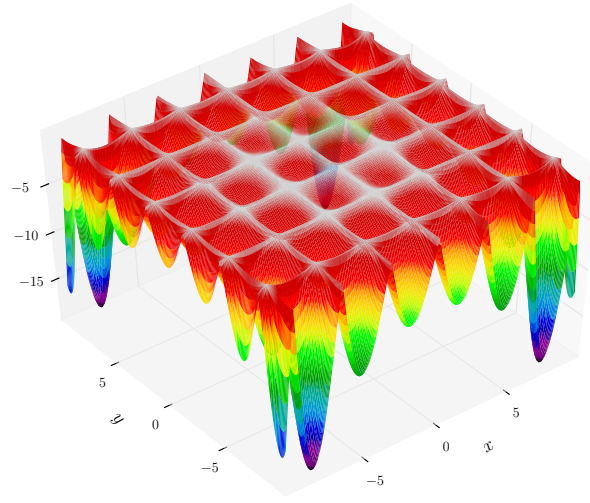
$$\begin{aligned}
 \text{minimize} \quad & f(x_1, x_2) = -(x_2 + 47) \sin \left(\sqrt{\left| \frac{x_1}{2} + (x_2 + 47) \right|} \right) - \\
 & - x_1 \sin(\sqrt{|x_1 - (x_2 + 47)|}), \\
 \text{domain} \quad & x_1 \in [-512, 512], \quad x_2 \in [-512, 512], \\
 \text{optimum} \quad & f(512, 404.2319) = -959.6407.
 \end{aligned} \tag{25}$$



Rysunek 4: Eggholder function

3.5 Hölder table function

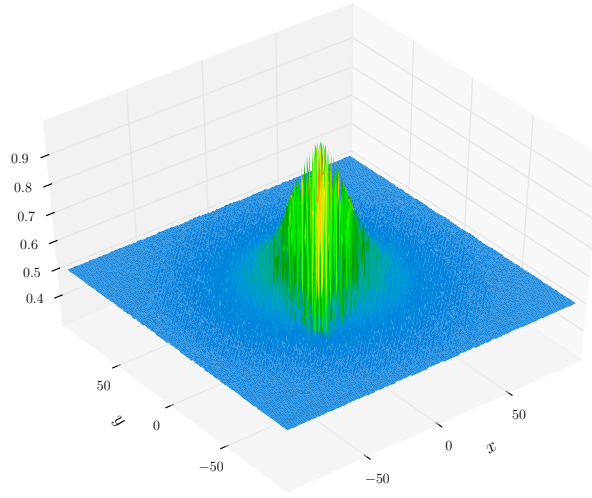
$$\begin{aligned}
 \text{minimize} \quad & f(x_1, x_2) = - \left| \sin(x_1) \cos(x_2) \exp \left(\left| 1 - \frac{\sqrt{x_1^2 + x_2^2}}{\pi} \right| \right) \right|, \\
 \text{domain} \quad & x_1 \in [-10, 10], \quad x_2 \in [-10, 10], \\
 \text{optima} \quad & f(8.05502, 9.66459) = -19.2085, \\
 & f(-8.05502, 9.66459) = -19.2085, \\
 & f(8.05502, -9.66459) = -19.2085, \\
 & f(-8.05502, -9.66459) = -19.2085.
 \end{aligned} \tag{26}$$



Rysunek 5: Hölder table function

3.6 Schaffer function N.4

$$\begin{aligned}
 \text{minimize} \quad & f(x_1, x_2) = 0.5 + \frac{\cos^2 [\sin (|x_1^2 - x_2^2|)] - 0.5}{[1 + 0.001(x_1^2 + x_2^2)]^2}, \\
 \text{domain} \quad & x_1 \in [-100, 100], \quad x_2 \in [-100, 100], \\
 \text{optima} \quad & f(0, 1.25313) = 0.292579, \\
 & f(0, -1.25313) = 0.292579.
 \end{aligned} \tag{27}$$



Rysunek 6: Schaffer function N.4

4 Funkcje ciągłe wielu zmiennych

Poniżej zawarto definicje problemów optymalizacji ciągłej n zmiennych (x_1 do x_n , tworzących razem wektor $\mathbf{x}$). Każdy opis zawiera wzór, dziedzinę (zakres zmiennych), rozwiązania optymalne (wraz z wartością funkcji celu) oraz wykres danej funkcji dla $n = 2$.

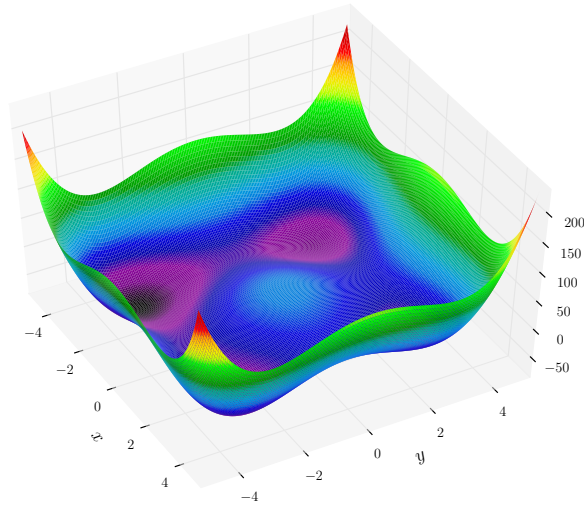
4.1 Styblinski–Tang function

$$\text{minimize} \quad f(\mathbf{x}) = f(x_1, x_2, \dots, x_n) = \frac{1}{2} \sum_{i=1}^n (x_i^4 - 16x_i^2 + 5x_i), \quad (28)$$

$$\text{domain} \quad x_i \in [-5, 5], \quad \text{dla } i = 1, 2, \dots, n,$$

$$\text{optimum} \quad -39.16617n \leq f(k_1, k_2, \dots, k_n) \leq -39.16616n, \quad (29)$$

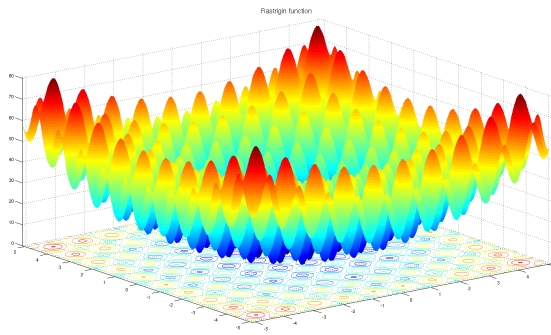
gdzie $k_1 = k_2 = \dots = k_n = -2.903534$.



Rysunek 7: Styblinski-Tang function ($n = 2$)

4.2 Rastrigin function

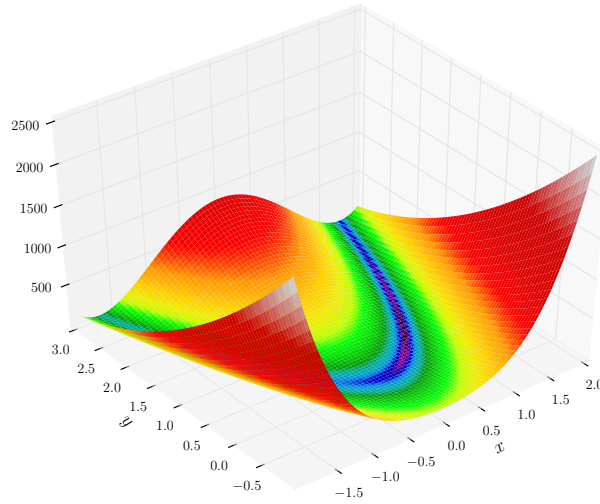
$$\begin{aligned}
 \text{minimize} \quad & f(x) = f(x_1, x_2, \dots, x_n) = 10n + \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i)], \\
 \text{domain} \quad & x_i \in [-5.12, 5.12], \quad \text{dla } i = 1, 2, \dots, n, \\
 \text{optimum} \quad & f(0, 0, \dots, 0) = 0.
 \end{aligned} \tag{30}$$



Rysunek 8: Rastrigin function ($n = 2$)

4.3 Rosenbrock function

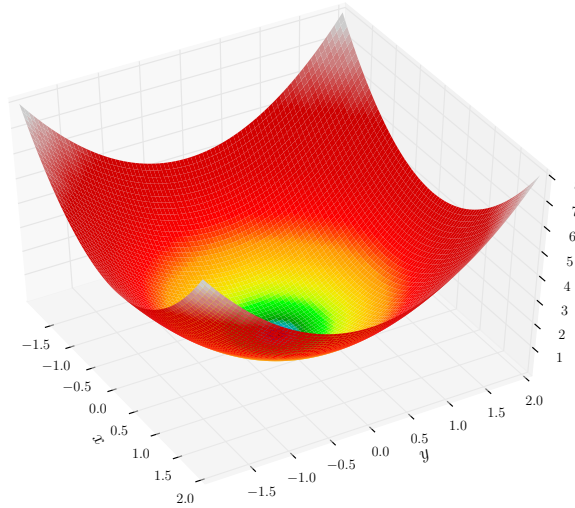
$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) = f(x_1, x_2, \dots, x_n) = \sum_{i=1}^{n-1} [100 (x_{i+1} - x_i^2)^2 + (1 - x_i)^2], \\ \text{domain} \quad & x_i \in [-100, 100], \quad \text{dla } i = 1, 2, \dots, n, \\ \text{optimum} \quad & f(1, 1, \dots, 1) = 0. \end{aligned} \tag{31}$$



Rysunek 9: Rosenbrock function ($n = 2$)

4.4 Sphere function

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) = f(x_1, x_2, \dots, x_n) = \sum_{i=1}^n x_i^2, \\ \text{domain} \quad & x_i \in [-100, 100], \quad \text{dla } i = 1, 2, \dots, n, \\ \text{optimum} \quad & f(0, 0, \dots, 0) = 0. \end{aligned} \tag{32}$$



Rysunek 10: Sphere function ($n = 2$)

5 Problemy ciągłe na bazie problemów dyskretnych

Poniżej zawarto definicje problemów optymalizacji ciągłej, które zostały utworzone poprzez pewną modyfikację problemów dyskretnych z rozdziału 2. Należy zwrócić uwagę, że choć podstawa problemów jest zwykle podobna, to optymalizowane są zupełnie inne elementy. Dzieje się tak dlatego, że rozwiązaniami problemów z rozdziału 2 były zbiory, permutacje itd. które są z definicji dyskretne, a nie ciągłe. Dlatego w problemach przedstawionych poniżej takie elementy są zwykle ustalone (np. założona kolejność odwiedzania miast dla problemu inspekcji), zaś ciągłe zmienne decyzyjne obejmują inne aspekty problemu.

5.1 Problem inspekcji

Problem inspekcji rozpatrujemy jako pewną modyfikację dyskretnego problemu komiwojażera. Dane jest $n \in \mathbb{N}_+$ kół tak że koło i -te przedstawione jest w postaci trójki (a_i, b_i, r_i) , gdzie $a_i \in \mathbb{R}_+$ oraz $b_i \in \mathbb{R}_+$ to współrzędne koła, zaś $r_i \in \mathbb{R}_+$ to jego promień. Oprócz tego dany jest punkt startowy $(x_0, y_0) \in \mathbb{R}_+^2$. Jako rozwiązanie należy podać n punktów od (x_1, y_1) do (x_n, y_n) tak aby:

1. Punkt $(x_i, y_i) \in \mathbb{R}_+^2$ znajdował się wewnątrz (lub na brzegu) koła i -tego (dla $i = 1, 2, \dots, n$).
2. Całkowita długość trasy (cyklu) pomiędzy punktami $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n \rightarrow 0$ była jak najmniejsza.

Problem ten modeluje sytuację, gdy dron startuje z bazy i ma za zadanie zrobić zdjęcia wszystkich pojazdów na parkingu, a do zrobienia zdjęcia potrzebuje znaleźć się odpowiednio blisko pojazdu.

Uwaga: domyślnym założeniem jest to że odległość między punktami jest liczona jako odległość euklidesowa (tzw. norma ℓ_2). Na niższą ocenę akceptowane są jednak również rozwiązania gdzie zamiast tego zostanie wykorzystana norma Manhattan (ℓ_1) lub norma Czebyszewa (ℓ_∞).

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :

- 2.1. $a_i \leftarrow \text{nextFloat}(10, 70)$.
- 2.2. $b_i \leftarrow \text{nextFloat}(10, 70)$.
- 2.3. $r_i \leftarrow \text{nextFloat}(1, 8)$.
3. $x_0 \leftarrow \text{nextFloat}(10, 70)$.
4. $y_0 \leftarrow \text{nextFloat}(10, 70)$.

5.2 Kwadratowe zagadnienie przydziału

Problem ten przedstawiamy jako pewną modyfikację dyskretnego kwadratowego zagadnienia przydziału. Dane jest n zakładów. Każdy zakład i ma dane koło określone przez trójkę (a_i, b_i, r_i) , gdzie $a_i \in \mathbb{R}_+$ oraz $b_i \in \mathbb{R}_+$ to współrzędne koła, a $r_i \in \mathbb{R}_+$ to jego promień. Ponadto dla każdej pary (i, j) zakładów dany jest przepływ $f_{i,j} \in \mathbb{R}_+$ (przy czym dla $i = j$ zakładamy $f_{i,j} = 0$). Jako rozwiązanie należy podać n punktów od (x_1, y_1) do (x_n, y_n) tak aby:

1. Punkt $(x_i, y_i) \in \mathbb{R}_+^2$ znajdował się wewnątrz (lub na brzegu) koła i -tego (dla $i = 1, 2, \dots, n$).
2. Wartość wyrażenia:

$$\sum_{i=1}^n \sum_{j=1}^n f_{i,j} \cdot d_{i,j} \quad (33)$$

była najmniejsza z możliwych, gdzie $d_{i,j}$ jest odległością punktów i oraz j .

Uwaga: domyślnym założeniem jest to że odległość między punktami jest liczona jako odległość euklidesowa (tzw. norma ℓ_2). Na niższą ocenę akceptowane są jednak również rozwiązania gdzie zamiast tego zostanie wykorzystana norma Manhattan (ℓ_1) lub norma Czebyszewa (ℓ_∞).

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. $a_i \leftarrow \text{nextFloat}(10, 70)$.
 - 2.2. $b_i \leftarrow \text{nextFloat}(10, 70)$.
 - 2.3. $r_i \leftarrow \text{nextFloat}(1, 8)$.
3. Dla i od 1 do n :
 - 3.1. Dla j od 1 do n :
 - 3.1.1. Jeśli $i = j$ to $f_{i,j} = 0$.
 - 3.1.2. Jeśli $i \neq j$ to $f_{i,j} \leftarrow \text{nextFloat}(1, 20)$.

5.3 Problem dwuplecakowy

Zadanie to jest modyfikacją dyskretnego problemu plecakowego. Dane jest $n \in \mathbb{N}_+$ przedmiotów. Przedmiot i opisany jest przez trójkę (c_i, w_i, v_i) , gdzie $c_i \in \mathbb{R}_+$ oznacza wartość przedmiotu, zaś $w_i \in \mathbb{R}_+$ oraz $v_i \in \mathbb{R}_+$ oznaczają ile miejsca przedmiot zająłby, gdyby w całości umieścić go w odpowiednio pierwszym i drugim plecaku. Oprócz tego dana jest wartość $B \in \mathbb{R}_+$ będąca pojemnością plecaków (oba mają taką samą). Należy dla każdego przedmiotu i określić dwie wartości ciągłe (x_i, y_i) , gdzie $x_i \in [0, 1]$ określa jaką część przedmiotu umieszczamy w plecaku pierwszym, zaś $y_i \in [0, 1]$ określa jaką część umieszczamy w plecaku drugim, tak aby:

1. $x_i + y_i \leq 1$ (dla $i = 1, 2, \dots, n$),
2. Jeśli przedmiot i został umieszczony w pierwszym plecaku to $x_i \geq 0.2$ (dla $i = 1, 2, \dots, n$),
3. Jeśli przedmiot i został umieszczony w drugim plecaku to $y_i \geq 0.2$ (dla $i = 1, 2, \dots, n$),
4. $\sum_{i=1}^n x_i w_i \leq B$,
5. $\sum_{i=1}^n y_i v_i \leq B$,
6. Wartość $\sum_{i=1}^n (x_i + y_i) c_i$ była maksymalna.

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 3. $S \leftarrow 0$.
 - 3.1. $c_i \leftarrow \text{nextFloat}(1, 10)$.
 - 3.2. $w_i \leftarrow \text{nextFloat}(1, 10)$.
 - 3.3. $v_i \leftarrow \text{nextFloat}(1, 10)$.
 - 3.4. $S \leftarrow S + (w_i + v_i)/2$.
4. $B \leftarrow \text{nextInt}(\lfloor \frac{S}{4} \rfloor, \lfloor \frac{S}{2} \rfloor)$.

5.4 Problem szeregowania na równoległych maszynach

Dane jest $n \in \mathbb{N}_+$ zadań oraz $m \in \mathbb{N}_+$ maszyn. Dla każdego zadania i dany jest jego bazowy czas wykonywania $p_i \in \mathbb{R}_+$ oraz maszyna na której ma zostać wykonane $a_i \in \{1, 2, \dots, m\}$. Należy określić wartości od s_1 do s_m , gdzie s_i to szybkość maszyny i , tak aby:

1. $s_i \geq 0.001$ (dla $i = 1, 2, \dots, m$),
2. $\sum_{i=1}^m s_i = 1$,
3. Czas zakończenia pracy maszyny która pracuje najdłużej był jak najmniejszy (tj. minimalizacja „maksymalnej” maszyny).

Faktyczny czas wykonania zadania i to p_i/s_{a_i} (czas bazowy podzielony przez szybkość maszyny na której go zaszeregowano). Zakładamy że maszyna wykonuje na raz co najwyżej jedno zadanie.

Generacja instancji Dla parametrów n i m oraz ziarna Z :

1. $\text{init}(Z)$.
2. Dla i od 1 do n :
 - 2.1. $p_i \leftarrow \text{nextFloat}(1, 20)$.
 - 2.2. $a_i \leftarrow \text{nextInt}(1, m)$.

6 Wielokryterialny permutacyjny problem przepływowy

Poniżej przedstawiono problem optymalizacji dyskretny dla laboratorium z optymalizacji wielokryterialnej. Podstawą problemu jest klasyczny problem przepływowy przedstawiony w podrozdziale 2.8 (dla kryterium makespan są to te same problemy). Zasadniczo dane wejściowe, ustalona liczba maszyn ($m = 3$), kolejność wykonywania zadań π oraz sposób obliczenia czasów zakończenia $C_{\pi(j)}^i$ (wraz z kolejnością ich obliczania) są prawie identyczne. Jediną różnicą jest to że w danych wejściowych dodatkowo dla każdego zadania j zdefiniowany jest pożądaný termin jego zakończenia (tzw. due date/deadline) oznaczany przez $d_j \in \mathbb{N}_+$, podobnie jak to miało miejsce dla problemu 2.7. Ze względu na to lekko zmienia się też sposób generacji instancji (pokazany w dalszej części).

Dla tego problemu można zdefiniować szereg kryteriów. W naszej sytuacji rozważamy następujące kryteria (wszystkie dotyczą minimalizacji):

1. Długość uszeregowania/czas zakończenia wszystkich zadań (makespan)

$$C_{\max} = \max_{j \in \{1, 2, \dots, n\}} C_j^3, \quad (34)$$

2. Suma czasów zakończenia (ściślej przepływów) wszystkich zadań (total flowtime)

$$\Sigma F = \sum_{j \in \{1, 2, \dots, n\}} C_j^3, \quad (35)$$

3. Maksymalne spóźnienie zadania (max tardiness)

$$T_{\max} = \max_{j \in \{1, 2, \dots, n\}} \max\{C_j^3 - d_j, 0\}, \quad (36)$$

4. Suma spóźnień zadań (total tardiness)

$$\Sigma T = \sum_{j \in \{1, 2, \dots, n\}} \max\{C_j^3 - d_j, 0\}, \quad (37)$$

5. Maksymalna nieterminowość zadania (max lateness)

$$L_{\max} = \max_{j \in \{1, 2, \dots, n\}} C_j^3 - d_j, \quad (38)$$

6. Suma nieterminowości zadań (total lateness)

$$\Sigma L = \sum_{j \in \{1, 2, \dots, n\}} C_j^3 - d_j. \quad (39)$$

Podczas laboratorium problem będzie definiowany z użyciem od 2 do 4 z powyższych kryteriów. Wszystkie kryteria do obliczenia wymagają tej samej macierzy czasów zakończenia C . Należy jednak uważać i rozróżniać kolejność danych wejściowych (zadanie j -te), a kolejność wg rozwiązania π (zadanie $\pi(j)$), czyli wykonywane jako j -te). Innymi słowy nie należy mylić C_j^i z $C_{\pi(j)}^i$. Jest to istotne dla kryteriów 3-6, gdyż wartości d_j zdefiniowane są wg kolejności danych wejściowych, a nie kolejności wykonywania. Tak więc można we wzorach (36)–(39) stosować $C_{\pi(j)}^i$ zamiast C_j^i , o ile d_j zostanie zamienione na $d_{\pi(j)}$. Dla wzorów (34) oraz (35) nie stanowi to żadnej różnicy (w praktyce wartością $C_{\max}$ jest i tak zawsze $C_{\pi(n)}^m$).

Generacja instancji Dla parametru n oraz ziarna Z :

1. $\text{init}(Z)$.
2. $S \leftarrow 0$.
3. Dla i od 1 do 3:
 - 3.1. Dla j od 1 do n :
 - 3.1.1. $p_j^i \leftarrow \text{nextInt}(1, 99)$.
 - 3.1.2. $S \leftarrow S + p_j^i$.
4. Dla j od 1 do n :
 - 4.1. $d_i \leftarrow \text{nextInt}(\lfloor \frac{S}{4} \rfloor, \lfloor \frac{S}{2} \rfloor)$.